
SEMINÁRIO DE PESQUISA

O Desafio da Alocação em Projetos de Software

Modelos de otimização para conciliar custos, prazos e competências de equipes.



Daniel Arruda de Santana e Luciano Trevisan Alberti

Universidade Estadual de Maringá – UEM

O Tamanho do Problema



CENÁRIO HIPOTÉTICO GERA ATÉ

10^{30}

Combinações possíveis.

O Cabo de Guerra da Alocação

Por que a tomada de decisão manual é inviável.



Decisões

- Qual profissional?
- Em qual projeto?
- Em qual período?



Restrições

- ✗ Disponibilidade do profissional.
- ✗ Capacidade máxima.
- ✗ Competências exigidas.
- ✗ Prazos rigorosos.



Objetivos

- ↘ **Minimizar**
Custo, duração, horas extras, ociosidade, risco e atrasos.
- ↗ **Maximizar**
Adequação de competências, utilização, produtividade, qualidade e prioridades.

Onde a complexidade aumenta!

A diferença entre o plano ideal e a realidade da execução.

Alocação Estática



Planejamento Prévio

- 📅 Atribuição fixa antes do início.
- 📖 Foco predominante na literatura.
- ⚠️ Rígida contra imprevistos reais.

Alocação Dinâmica



Adaptação Contínua

- 🔄 Reprogramação em tempo de execução.
- ✂️ Re-otimização constante baseada em:
 - Indisponibilidade inesperada.
 - Atrasos de tarefas.
 - Novas demandas e prioridades.

Mapeamento da Literatura

As principais abordagens recentes para o problema.

2011 • Ren et al.



Cooperative
Co-evolutionary
Optimization of Software
Project Staff Assignments...

2013 • Stylianou et al.



A multi-objective genetic
algorithm for intelligent
software project
scheduling...



ALGORITMO GENÉTICO

*O algoritmo genético é apenas o
mecanismo. A modelagem do
problema é onde as diferenças
aparecem.*



2022 • Costa et al.

A Genetic Algorithm-Based
Approach to Support
Forming Multiple Scrum
Project Teams



2025 • Cunha et al.

A Genetic Algorithm with
Convex Combination
Crossover for Software
Team...

A Lente de Análise

Para transformar o problema em busca, precisamos decidir:

1

O que é uma
solução?



2

Como
representá-la?



3

Como saber se ela é
boa?



4

Como gerar novas
soluções?



Essa é a nossa lente de análise.

Para cada artigo vamos responder essas quatro perguntas

O Cromossomo Genérico

Transformando decisões de alocação em código computacional.



➡ Cada gene representa uma decisão de alocação.

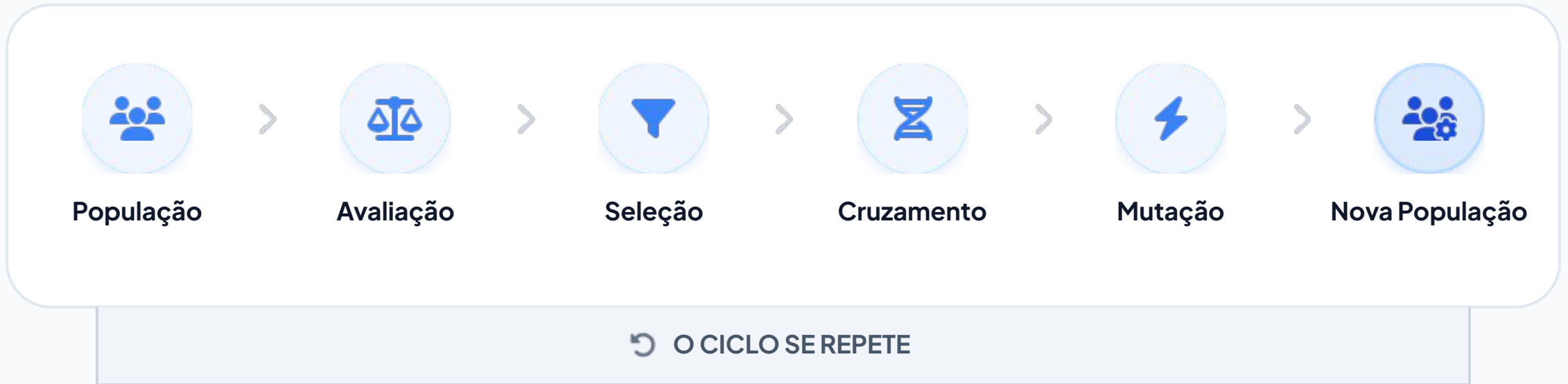
Todos os artigos utilizam este mesmo formato de cromossomo?

NÃO.

Vamos demonstrar nos 4 casos.

O Ciclo Evolutivo

O motor por trás dos algoritmos genéticos.



**O que muda de um artigo para outro não é apenas o algoritmo.
É principalmente o que cada etapa significa.**

2011 • Ren et al.

Dividir para conquistar: a separação do problema.

ALGORITMO EVOLUTIVO COOPERATIVO
VS
GA DE POPULAÇÃO ÚNICA



Subproblema 1: Alocação

Atribuição de profissionais em equipes.

- ✓ Foco nos recursos humanos.
- ✓ Garante mínimo de membros.



Subproblema 2: Agendamento

Ordenação e distribuição de tarefas.

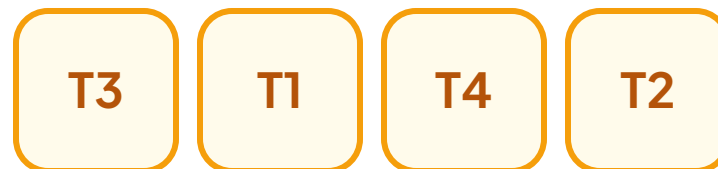
- ✓ Foco nas dependências temporais.
- ✓ Respeito à precedência de pacotes.

1 2 A Representação do CCEA

Duas populações evoluindo em paralelo, com cromossomos diferentes.

População 1

Cromossomo de Tarefas



Gene = Ordem de execução.

População 2

Cromossomo de Pessoas



Gene = ID da Equipe atribuída.

3

4

A Avaliação Simbiótica

O cruzamento é isolado, mas a avaliação exige cooperação.



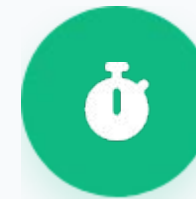
Indivíduo Alvo

(Ex: Cromossomo da População de Agendamento)



O Melhor da Outra Pop.

(Ex: O melhor arranjo de Equipes atual)



Fitness (Avaliação)

Tempo total de conclusão do projeto simulado.



Geração: Cruzamento e mutação tradicionais.



Isolamento: Populações não trocam genes entre si.

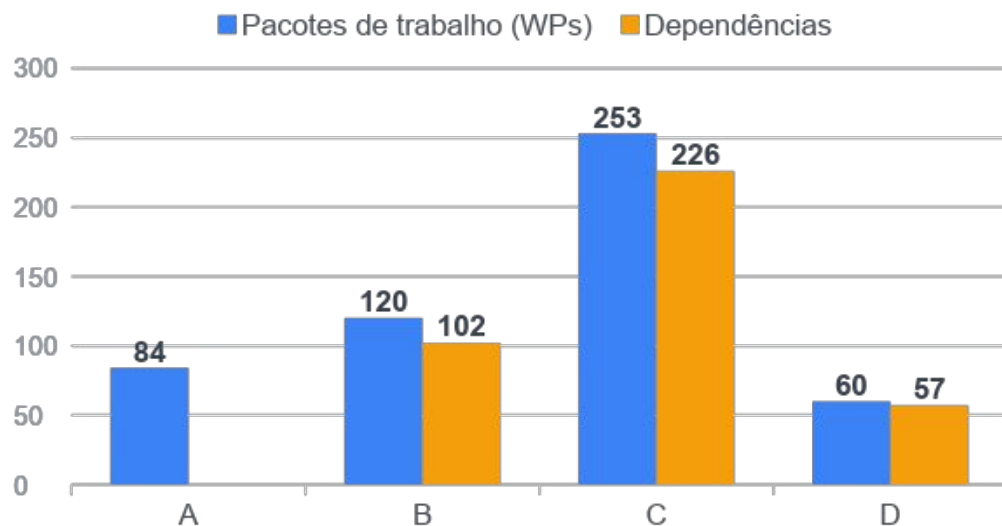


Ren et al. • O Experimento

Quatro projetos reais e uma pergunta: quanto as duas populações devem conversar?

4 projetos industriais (4 empresas)

Tamanho e nº de dependências entre pacotes de trabalho



A manutenção Y2K (sem dependências) **B** sistema multiplataforma
C venda online de TI **D** cotação → pedido (Canadá)

3 configurações, mesmo orçamento

Gerações internas × externas (trocas de referência)

Config. I

1 × 100



troca a cada geração

Config. II

10 × 10



troca a cada 10 gerações

Config. III

100 × 1



uma única troca = GA sem coevolução

evolui equipes (TC)

evolui ordem dos pacotes (WPO)

50 pessoas · 5 equipes · 30 execuções por configuração e projeto



Ren et al. • Conclusão

A coevolução vence o GA tradicional.

Quem achou o projeto mais curto?

1º

Config. I e II (CCEA)



empate estatístico entre elas

2º

Config. III (GA população única)

pior que os CCEAs nos 4 projetos

3º

Busca aleatória

mesmo com o dobro de soluções geradas

$$p < 0,000003$$

CCEA vs GA único em todos os projetos (Wilcoxon + Holm)

Config. I

mais eficiente: melhores soluções com menos avaliações



Exceção: Projeto A. Sem dependências, quase não há espera entre pacotes — a coevolução ganha pouco na prática.

Limites: pessoas idênticas, uma equipe por pessoa e overhead de comunicação ignorado.

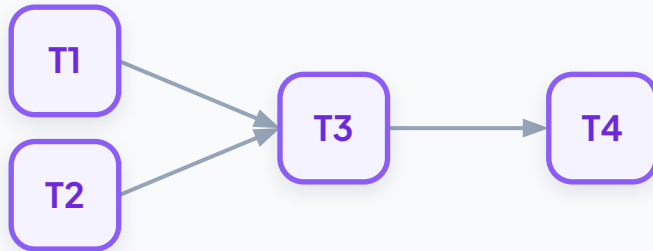
2013 • Stylianou & Andreou

GA MULTIOBJETIVO • NSGA-II

Agendar tarefas e montar a equipe ao mesmo tempo, com pessoas diferentes entre si.

O que entra no modelo

Tarefas com dependências (fim → início)



Desenvolvedores com experiência por habilidade [0..1]



Habilidade S2. Diferente de Ren: pessoas NÃO são intercambiáveis.

Três objetivos em conflito



Duração

MINIMIZAR

Minimiza os dias ociosos entre uma tarefa e sua predecessora.



Experiência

MAXIMIZAR

Soma o máximo + a média da experiência dos alocados em cada habilidade.



Tamanho da equipe

MINIMIZAR

Menos pessoas por tarefa → menos esforço e comunicação.

Restrições (tornam a solução inviável):

Dependência violada

Habilidade não coberta

Conflito de alocação

1

2

O Cromossomo Misto

Uma única população: cada tarefa carrega quando começa e quem a executa.



Exemplo da Fig. 1 do artigo: 4 tarefas, 5 desenvolvedores. T1 começa no dia 1 com E1 e E3; T2 no dia 11 com E4. (T3 e T4 ilustrativos.)



Parte inteira → cronograma

Gene inteiro positivo: o dia em que a tarefa começa.



Parte binária → equipe

Um bit por desenvolvedor: 1 = está na tarefa, 0 = não está.



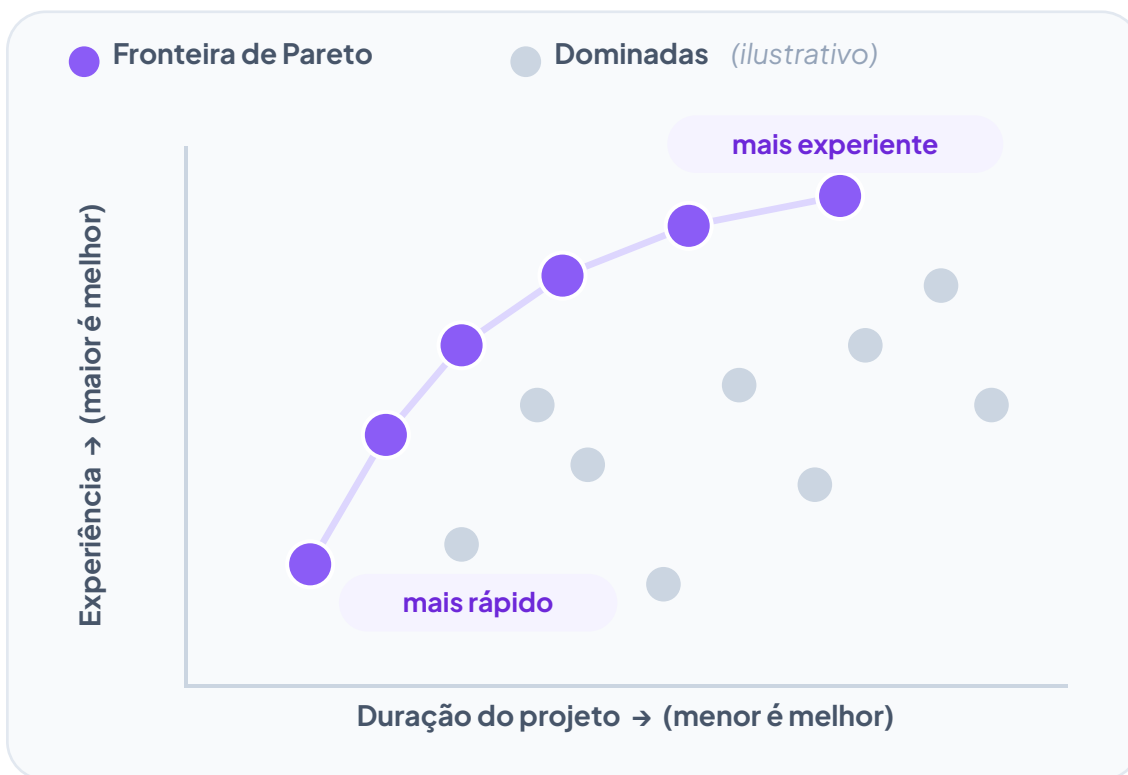
População inicial

100 indivíduos com dias e alocações aleatórios; 15 execuções × 3.500 gerações.

3

Avaliação: Não Existe “O Melhor”

O NSGA-II devolve uma frente de soluções de compromisso — a fronteira de Pareto.



Exemplo real do artigo (Cenário 1)

E2 tem 0,82 na habilidade de T2; E3 tem só 0,36. Mas E2 também é o melhor para T3.



Solução A E3 faz T2 (0,36)

T2 e T3 em paralelo → projeto mais curto



Solução B E2 faz T2 (0,82)

T3 espera E2 → projeto mais longo, equipe melhor

As duas são ótimas — cada uma em um critério.

1. Viável vence inviável

2. Menos violações vence

3. Frente de não-dominância

4. Diversidade (crowding)

4

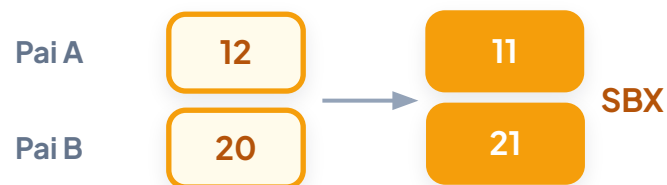
Cruzamento, Mutação... e Coevolução?

Cada parte do cromossomo tem o seu próprio operador.

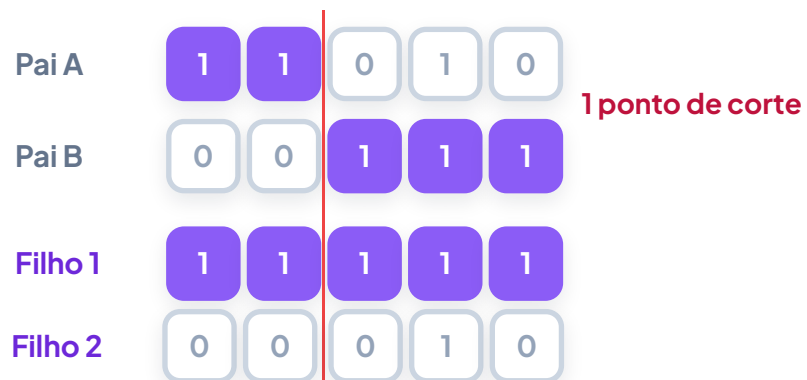


Cruzamento

Dias de início (inteiros)



Alocação (bits)



Probabilidade de recombinar: 0,90 por tarefa



Mutação

Dias de início



Alocação



Taxa: $2 / n^{\circ}$ de tarefas.

Seleção: torneio de 4 + elitismo.

Coevolução?

NÃO.

Uma só população. A “cooperação” aqui acontece entre objetivos: **a fronteira de Pareto equilibra duração, experiência e tamanho**, e o gerente escolhe a solução que prefere.

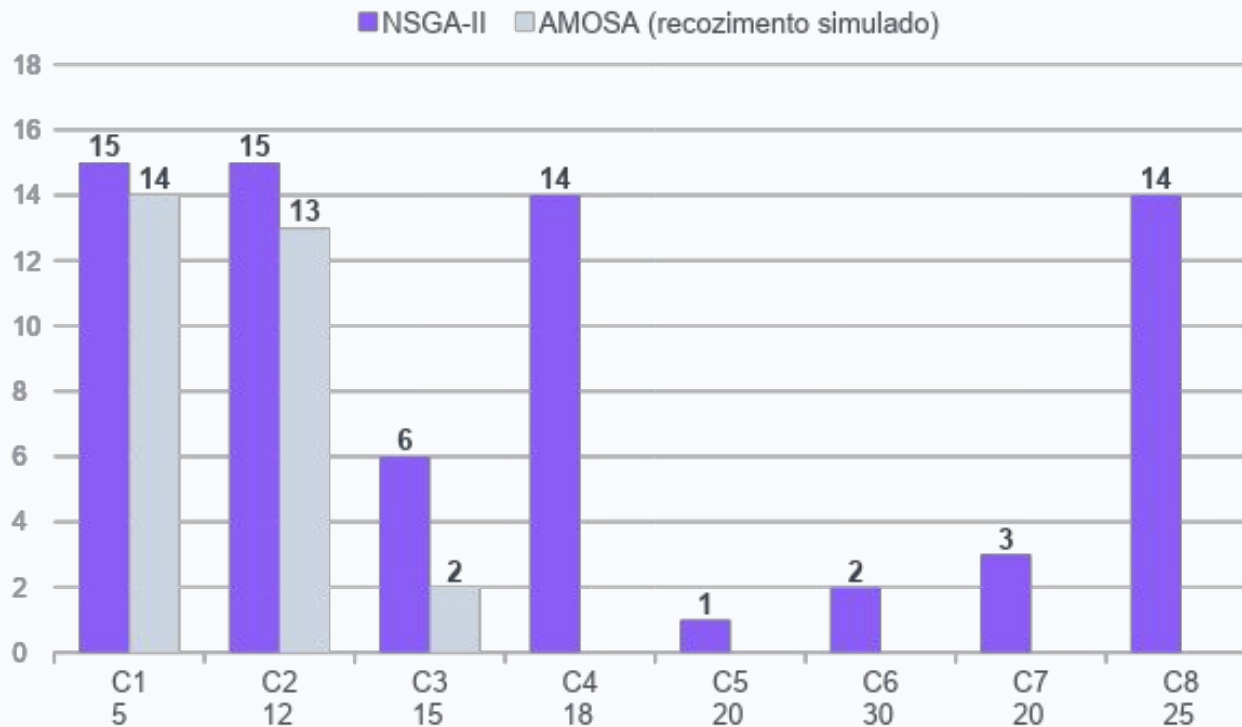


Stylianou & Andreou • Conclusão

Ótimo para projetos pequenos e médios; trava quando o projeto cresce.

Execuções que terminaram com soluções viáveis (de 15)

8 cenários sintéticos, criados com gerentes de projeto; abaixo, nº de tarefas



- ✓ Pequenos e médios: soluções viáveis e ótimas — no C1, 13 de 15 execuções acharam o ótimo.
- ✗ Muitas tarefas + muitas dependências + poucos devs → violações de dependência (C5–C7).
- ! Soluções quase ótimas: “buracos” desnecessários no cronograma.
- ! Validação só com cenários sintéticos; dados reais ficaram para trabalhos futuros.

NSGA-II supera o AMOSA e entrega um cardápio de soluções ao gerente.

2022 • Costa et al.

GA • VÁRIOS TIMES SCRUM AO MESMO TEMPO

Antes de otimizar, é preciso medir: transformar tarefas Scrum em perfis de competência.



Problema MTF: vários projetos simultâneos disputam o mesmo grupo de desenvolvedores. Objetivo: a melhor formação **global**.

Como a matriz é calculada

Para cada tarefa do projeto: $(1 + \text{similaridade Manhattan entre etiquetas}) \times (1 + \text{nº de vezes que fez a tarefa padrão})$; depois, a média.

1

2

O Cromossomo de Permutação

Um único cromossomo descreve todos os times de todos os projetos.

Posição = vaga num projeto Gene = ID do desenvolvedor



Fenótipo (Fig. 3 do artigo): $A = \{05, 02, 09, 11, 01\}$ · $B = \{08, 04, 07\}$ · $C = \{12, 03, 10, 06\}$



Restrição de graça

É uma permutação: nenhum dev aparece duas vezes — ninguém fica em dois times.



Inverso de Ren

Ren: gene = equipe de cada pessoa.
Costa: gene = pessoa de cada vaga.



Uma população

100 cromossomos; cada um é uma formação global completa.

3

4

Avaliação e Geração

Dois fitness à escolha do gerente; PMX para cruzar sem quebrar a permutação.



f1 · Alocação por Redundância

Média da competência de cada dev para o projeto da sua vaga.

Times multifuncionais com sobreposição: um membro cobre o outro.



f2 · Alocação Especializada

Mesma média, mas o gerente aumenta o peso de tags críticas (até x5).

Ex.: exigir TypeScript e MySQL sem perder a visão multifuncional.

Coevolução? Não. A “cooperação” é entre projetos: um só cromossomo resolve a disputa por pessoas de forma global.



PMX · Partially Mapped Crossover

Troca um trecho e conserta repetições por mapeamento

Pai 1

1

2

3

4

5

6

7

8

Pai 2

4

5

6

7

8

1

2

3

Filho

1

2

6

7

8

3

4

5

Trecho do Pai 2 + resto do Pai 1. Repetidos 6, 7, 8 viram **3, 4, 5** pelo mapeamento.

Cruzamento

PMX · 50%

Mutação

Swap · 5%

Seleção

Torneio (3)

Elitismo

1 indivíduo

Parada

5 ger.
estáveis



Costa et al. • Conclusão

O GA monta times parecidos com os dos gerentes — e ajuda a resolver a disputa entre eles.

85%

de precisão

times do GA vs. times montados por 4 gerentes

75%

de aceitação

quando os gerentes precisaram dividir as mesmas pessoas

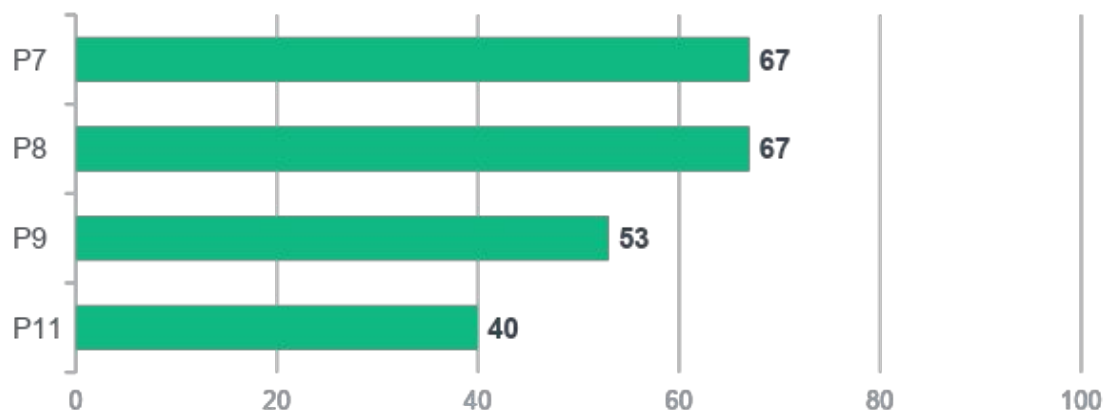
52

desenvolvedores

12 projetos reais · 1.063 tarefas estruturadas

Gerentes disputando as mesmas pessoas

Taxa média de conflito por projeto (cenário compartilhado)



Limitações

- ! Partida a frio: sem histórico de projetos, não há perfil.
- ! Tarefas iguais são tratadas como de mesmo esforço.
- ! Só projetos web e mobile, de uma única empresa.
- ✗ Avalia só competência técnica — quem trabalha bem junto?

2025 • Cunha et al.

GA • CRUZAMENTO POR COMBINAÇÃO CONVEXA

Competência técnica e histórico de colaboração na mesma conta — um time por vez.

Função objetivo (maximizar)

$$F_{\text{total}} = F_{\text{base}} \times P_{\text{collab}}$$

F_{base}: aptidão técnica: similaridade Manhattan entre skills do dev e do projeto

P_{collab}: aptidão colaborativa: pesos do grafo de colaboração

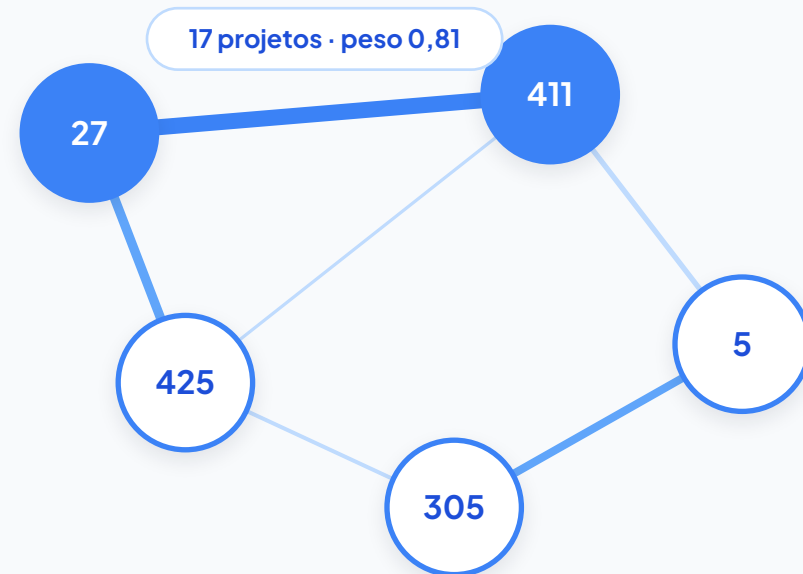
Restrições do modelo

- ✓ Cobrir todas as skills exigidas
- ✓ Tamanho entre mín. e máx.
- ✓ Histórico colaborativo ≥ 0
- ✓ Cada dev em um só time
- ✓ Skills normalizadas [0..1]

Decisão com especialista: gerentes preferem formar UM time por vez

Grafo de colaboração

Nós = desenvolvedores · arestas = projetos já feitos juntos



Peso = nº de projetos juntos × (0,3·SLF + 0,3·OSF + 0,4·SLF·OSF)
SLF = liderança (pesquisa) · OSF = sucesso do projeto (NPS)

1

2

Cromossomo = Um Time

Cada indivíduo é uma configuração candidata para o mesmo projeto.

População de times candidatos

Gene = desenvolvedor · tamanho = vagas (em geral 4)

#1	27	411	5	305	👑
#2	27	425	18	142	
#3	363	359	5	411	
#4	290	138	425	27	
#5	154	205	351	105	

Dados: 149 desenvolvedores · 47 projetos reais

O ciclo do GA aprimorado (100 gerações)



1. Avaliar

$F = \text{técnica} \times \text{colaboração}$



2. Elitismo

os melhores 25% passam



3. Torneio

escolhe os pais



4. Cruzamento convexo

mistura ponderada



5. Mutação

taxa 0,005 · tira duplicatas



6. Injeção

1 indivíduo novo e único



Injeção + mutação mantêm a diversidade e evitam estagnação

3

4

O Cruzamento por Combinação Convexa

Em vez de recortar e colar genes, o filho é uma mistura ponderada dos pais.

$$\text{filho1} = \alpha \cdot \text{pai1} + (1 - \alpha) \cdot \text{pai2} \quad \alpha \sim U(0, 1)$$

Pai 1

12

40

7

88

Pai 2

30

10

55

20

Filho 1

17

31

21

68

$\alpha = 0,7$
(exemplo)

$$0,7 \times 12 + 0,3 \times 30 \\ = 17,4 \rightarrow 17$$

Duplicatas $\rightarrow$ mutação · skill faltando $\rightarrow$ troca de gene. Arredondamento ilustrativo.

PMX / 1 ponto

Filhos só recombina genes que já existem nos pais $\rightarrow$ menos diversidade, convergência prematura.

Combinação convexa

Pode criar genes “entre” os pais $\rightarrow$ explora regiões novas do espaço de busca.



Coevolução? Não. Mas a cooperação entra no fitness: o GA premia quem já colaborou bem — **cooperação** entre PESSOAS, não entre populações.

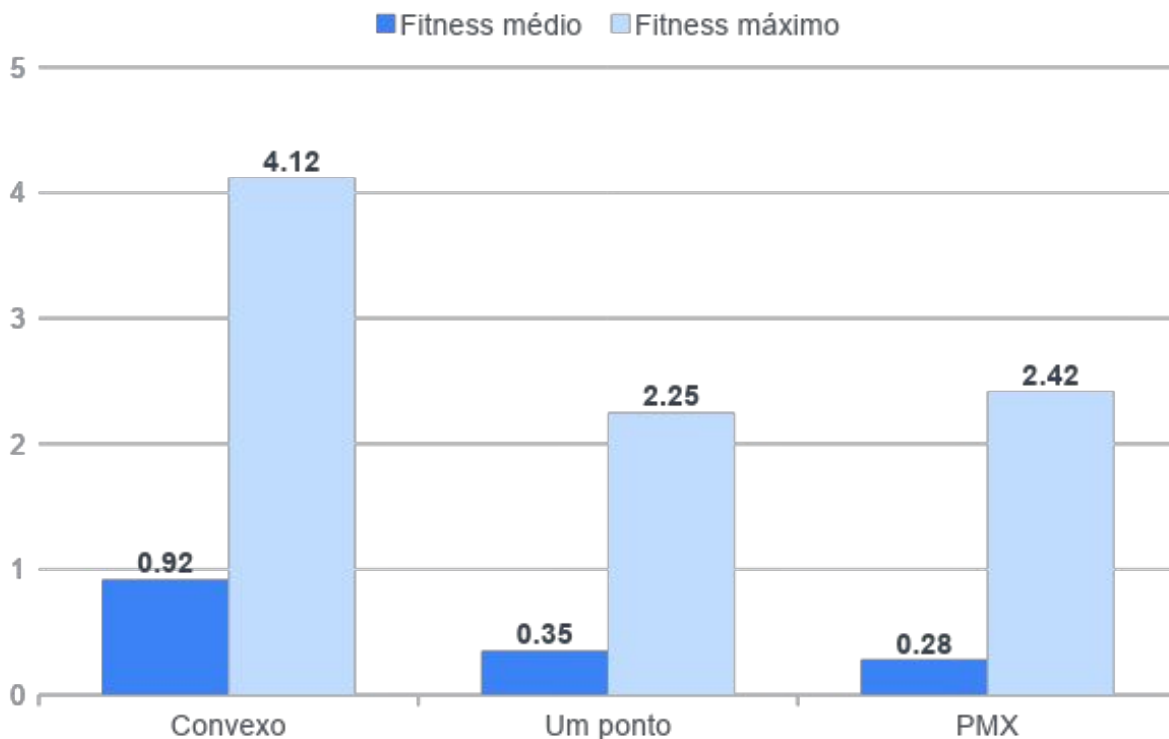


Cunha et al. • Conclusão

O cruzamento convexo encontra times melhores — e o grafo captura bem a colaboração.

Fitness dos times por operador de cruzamento

40 rodadas · 100 gerações · times de 4 pessoas



2,6x

fitness médio maior que o cruzamento de um ponto (3,3x vs PMX)

p = 0,001

convexo vence os outros dois (Friedman + Nemenyi); PMX ≈ um ponto

- ✓ Grafo validado: 12 cenários, 789 relações entre devs
- ! Uma só organização; um só objetivo
- ! Alta variação (desvio-padrão 1,05)
- ✗ Forma um time por vez — vários juntos fica para o futuro

A Lente Aplicada aos 4 Artigos

Mesmo mecanismo evolutivo; modelagens muito diferentes.

	2011 · Ren	2013 · Stylianou	2022 · Costa	2025 · Cunha
1 O que é uma solução?	Ordem dos pacotes + equipe de cada pessoa	Cronograma + equipe de cada tarefa	Todos os times de todos os projetos	Um time para um projeto
2 Como representar?	2 populações: ordem dos pacotes · pessoa → equipe	1 população: dia de início + bits	Permutação: vaga → dev	Lista de devs do time
3 Como avaliar?	Tempo de conclusão simulado com a outra população	3 objetivos via Pareto (NSGA-II)	Competência média (f1) ou ponderada (f2)	Técnica x colaboração (grafo)
4 Como gerar?	1 ponto + troca de posição	SBX + 1 ponto · polinomial + bit	PMX + troca (swap)	Combinação convexa + injeção
✓ Validação	4 projetos industriais	8 cenários sintéticos	12 projetos, 4 gerentes: 85%	47 projetos, 149 devs